

```
// Full-stack MERN Application for Elysian bites Restaurant Digital Transformation
// Enhanced with Ordering Interface, Reservation System, Auth, Tracking, Payments, Admin
Dashboard, Reviews, Loyalty
```

```
////////////////////////////////////
```

```
// server/config/mailer.js
```

```
////////////////////////////////////
```

```
const nodemailer = require('nodemailer');
const transporter = nodemailer.createTransport({
  host: process.env.SMTP_HOST,
  port: process.env.SMTP_PORT,
  auth: { user: process.env.SMTP_USER, pass: process.env.SMTP_PASS }
});
module.exports = transporter;
```

```
////////////////////////////////////
```

```
// server/models/User.js
```

```
////////////////////////////////////
```

```
const mongoose = require('mongoose');
const bcrypt = require('bcrypt');

const UserSchema = new mongoose.Schema({
  name: String,
  email: { type: String, unique: true, required: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['customer','admin'], default: 'customer' },
  emailVerified: { type: Boolean, default: false },
  loyaltyPoints: { type: Number, default: 0 },
  createdAt: { type: Date, default: Date.now }
});
UserSchema.pre('save', async function(next) {
```

```

if (!this.isModified('password')) return next();

this.password = await bcrypt.hash(this.password, 12);

next();

});

UserSchema.methods.comparePassword = function(candidate) { return
bcrypt.compare(candidate,this.password); };
module.exports = mongoose.model('User', UserSchema);

////////////////////////////////////
// server/models/Review.js
////////////////////////////////////

const ReviewSchema = new mongoose.Schema({

  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  rating: { type: Number, min:1, max:5 },
  comment: String,
  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Review', ReviewSchema);

////////////////////////////////////
// server/models/Reservation.js (updated)
////////////////////////////////////

const ReservationSchema = new mongoose.Schema({

  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  date: Date,
  time: String,
  partySize: Number,
  tableNumber: Number,
  status: { type: String, enum: ['confirmed','waiting','cancelled'], default: 'confirmed' },
  paymentIntentId: String,
  createdAt: { type: Date, default: Date.now }

```

```

});

module.exports = mongoose.model('Reservation', ReservationSchema);

////////////////////////////////////
// server/models/Order.js (updated)
////////////////////////////////////

const OrderSchema = new mongoose.Schema({

  user: { type: mongoose.Schema.Types.ObjectId, ref:'User' },

  items: [{ menuItem: { type: mongoose.Schema.Types.ObjectId, ref:'MenuItem' }, quantity:Number,
price:Number }],

  total: Number,

  status: { type: String, enum:['pending','preparing','served','completed'], default:'pending' },

  paymentIntentId: String,

  createdAt: { type: Date, default: Date.now }

});

module.exports = mongoose.model('Order', OrderSchema);

////////////////////////////////////
// server/controllers/authController.js
////////////////////////////////////

const jwt = require('jsonwebtoken');
const User = require('../models/User');
const transporter = require('../config/mailer');

exports.register = async (req,res) => {

  const { name,email,password } = req.body;

  const user = new User({name,email,password}); await user.save();

  const token = jwt.sign({id:user._id},process.env.JWT_SECRET,{expiresIn:'1d'});

  const url = `${process.env.FRONTEND_URL}/verify/${token}`;

  await transporter.sendMail({to:email,subject:'Verify Email',html:`Click <a href="${url}">here</a> to
verify.`});

  res.status(201).json({message:'Verify your email'});

```

```

};

exports.verifyEmail = async (req,res) => {

  const {token}= req.params; const {id}=jwt.verify(token,process.env.JWT_SECRET);

  await User.findByIdAndUpdate(id,{emailVerified:true}); res.json({message:'Email verified'});

};

exports.login = async (req,res) => {

  const {email,password}=req.body; const user=await User.findOne({email});

  if(!user || !(await user.comparePassword(password))) return res.status(401).json({error:'Invalid'});

  if(!user.emailVerified) return res.status(401).json({error:'Verify Email'});

  const token=jwt.sign({id:user._id,role:user.role},process.env.JWT_SECRET,{expiresIn:'7d'});

  res.json({token,user:{id:user._id,name:user.name,email:user.email,role:user.role,points:user.loyaltyPoints}});

};

////////////////////////////////////

// server/controllers/bookingController.js

////////////////////////////////////

const Reservation = require('../models/Reservation');

const Table = require('../models/Table');

const transporterBooking = require('../config/mailer');

exports.checkAvailability = async (req,res) => {

  const {date,time,partySize}=req.query;

  const tables=await Table.find();

  const booked=await Reservation.find({date:new Date(date),time,status:'confirmed'});

  const reservedNums=booked.map(r=>r.tableNumber);

  const available=tables.filter(t=>!reservedNums.includes(t.number)&&t.capacity>=partySize);

  res.json(available);

};

exports.createBooking = async (req,res) => {

  const {userId, date,time,partySize,tableNumber} = req.body;

```

```

let status = 'confirmed';

const exists = await Reservation.findOne({date,new
Date(date),time,tableNumber,status:'confirmed'});

if (exists) status='waiting';

const booking=new Reservation({user:userId,date,time,partySize,tableNumber,status});

await booking.save();

await transporterBooking.sendMail({to:req.body.email,subject:'Booking '+status,html:`Your
booking is ${status}`});

res.status(201).json(booking);
};

////////////////////////////////////
// server/controllers/orderController.js
////////////////////////////////////

const Order = require('../models/Order');
const stripe = require('stripe')(process.env.STRIPE_SECRET_KEY);
const transporterOrder = require('../config/mailer');

exports.createOrder = async (req,res) => {

const {userId,items,email}=req.body;

const total=items.reduce((sum,i)=>sum+i.price*i.quantity,0)*100;

const pi=await stripe.paymentIntents.create({amount:total,currency:'usd',receipt_email:email});

const order=new Order({user:userId,items,total:total/100,paymentIntentId:pi.id}); await
order.save();

await transporterOrder.sendMail({to:email,subject:'Order Placed',html:`Order ${order._id}
placed.`});

res.json({clientSecret:pi.client_secret});
};

exports.trackOrder = async (req,res) => {

const {orderId}=req.params; const order=await Order.findById(orderId);

res.json({status:order.status});
};

```

```

////////////////////
// server/controllers/reviewController.js
////////////////////

const Review = require('../models/Review');
exports.addReview = async (req,res) => {

  const {userId,rating,comment}=req.body;
  const rev=new Review({user:userId,rating,comment}); await rev.save();
  res.status(201).json(rev);
};

exports.getReviews = async (req,res) => {

  const reviews = await Review.find().populate('user','name'); res.json(reviews);
};

////////////////////
// server/controllers/loyaltyController.js
////////////////////

const User = require('../models/User');
exports.getPoints = async (req,res) => {

  const {userId}=req.params; const user=await User.findById(userId);
  res.json({points:user.loyaltyPoints});
};

exports.redeemCoupon = async (req,res) => {

  const {userId,cost}=req.body; const user=await User.findById(userId);
  if(user.loyaltyPoints<cost) return res.status(400).json({error:'Insufficient points'});
  user.loyaltyPoints-=cost; await user.save();
  res.json({message:'Coupon redeemed'});
};

////////////////////
// server/routes/authRoutes.js

```

```

////////////////////////////////////
const express = require('express'); const router=express.Router();
const {register,verifyEmail,login} = require('../controllers/authController');
router.post('/register',register);
router.get('/verify/:token',verifyEmail);
router.post('/login',login);
module.exports = router;

////////////////////////////////////
// server/routes/bookingRoutes.js
////////////////////////////////////
const express=require('express'); const router=express.Router();
const {checkAvailability,createBooking}=require('../controllers/bookingController');
router.get('/availability',checkAvailability);
router.post('/',createBooking);
module.exports=router;

////////////////////////////////////
// server/routes/orderRoutes.js
////////////////////////////////////
const express=require('express'); const router=express.Router();
const {createOrder,trackOrder}=require('../controllers/orderController');
router.post('/',createOrder);
router.get('/:orderId/status',trackOrder);
module.exports=router;

////////////////////////////////////
// server/routes/reviewRoutes.js
////////////////////////////////////
const express=require('express'); const router=express.Router();
const {addReview,getReviews}=require('../controllers/reviewController');

```

```

router.post('/',addReview);

router.get('/',getReviews);

module.exports=router;

////////////////////////////////////
// server/routes/loyaltyRoutes.js
////////////////////////////////////

const express=require('express'); const router=express.Router();

const {getPoints,redeemCoupon}=require('../controllers/loyaltyController');

router.get('/:userId/points',getPoints);

router.post('/redeem',redeemCoupon);

module.exports=router;

```

```

////////////////////////////////////
// server/server.js (mount routes)
////////////////////////////////////

require('dotenv').config();

const express = require('express');

const connectDB = require('./config/db');

const app = express(); app.use(express.json()); connectDB();

app.use('/api/auth',require('./routes/authRoutes'));

app.use('/api/booking',require('./routes/bookingRoutes'));

app.use('/api/orders',require('./routes/orderRoutes'));

app.use('/api/reviews',require('./routes/reviewRoutes'));

app.use('/api/loyalty',require('./routes/loyaltyRoutes'));

app.listen(process.env.PORT || 5000,()=>console.log('Server running'));

```

```

////////////////////////////////////
// client/src/components/MenuDisplay.js
////////////////////////////////////

import React, {useEffect,useState} from 'react';

```



```

import { fetchMenu } from '../services/api';

export default function MenuDisplay(){

  const [menu,setMenu]=useState([]);

  useEffect(()=>{fetchMenu().then(res=>setMenu(res.data));},[]);

  return (<div className="grid grid-cols-3 gap-4">{menu.map(item=>(<div key={item._id}
className="p-4 border
rounded"><h3>{item.name}</h3><p>{item.description}</p><p>${item.price}</p></div>))}</div>);
}

////////////////////////////////////

// client/src/components/CustomizationModal.js

////////////////////////////////////

import React,{useState} from 'react';

export default function CustomizationModal({item,onAdd}){

  const [notes,setNotes]=useState(""); const [qty,setQty]=useState(1);

  return (<div className="modal"><h2>Customize {item.name}</h2><textarea value={notes}
onChange={e=>setNotes(e.target.value)} placeholder="Special requests"/><input type="number"
value={qty} onChange={e=>setQty(e.target.value)} min={1}/><button
onClick={()=>onAdd(item,qty,notes)}>Add</button></div>);
}

////////////////////////////////////

// client/src/components/Cart.js

////////////////////////////////////

import React from 'react';

export default function Cart({cart,onRemove}){

  return (<div className="cart"><h2>Your Cart</h2>{cart.map((ci,i)=>(<div key={i} className="flex
justify-between"><p>{ci.qty}x {ci.item.name}</p><button
onClick={()=>onRemove(i)}>Remove</button></div>))}</div>);
}

////////////////////////////////////

// client/src/components/SpecialRequests.js

```

```

////////////////////////////////////
import React,{useState} from 'react';

export default function SpecialRequests({onSave}){const
[req,setReq]=useState("");return(<div><h3>Any special requests?</h3><textarea value={req}
onChange={e=>setReq(e.target.value)}/><button onClick={()=>onSave(req)}>Save</button></div>;)}

////////////////////////////////////

// client/src/components/AvailabilityChecker.js

////////////////////////////////////

import React,{useState} from 'react';

import axios from 'axios';

export default function AvailabilityChecker(){

  const [date,time,party,available]=[useState(""),useState(""),useState(1),useState([])];

  const
check=()=>axios.get(`/api/booking/availability?date=${date}&time=${time}&partySize=${party}`).the
n(r=>available[1](r.data));

  return (<div><input type="date" value={date[0]} onChange={e=>date[1](e.target.value)}/><input
type="time" value={time[0]} onChange={e=>time[1](e.target.value)}/><input type="number"
value={party[0]} onChange={e=>party[1](e.target.value)}/><button
onClick={check}>Check</button><ul>{available[0].map(t=><li key={t.number}>Table {t.number} -
seats {t.capacity}</li>)}</ul></div>;

}

////////////////////////////////////

// client/src/components/BookingConfirmation.js

////////////////////////////////////

import React from 'react';

export default function BookingConfirmation({booking}){

  return (<div><h2>Booking {booking.status}</h2><p>Table {booking.tableNumber} on {new
Date(booking.date).toLocaleDateString()} at {booking.time}</p></div>;

}

////////////////////////////////////

// client/src/components/WaitlistManager.js

```

```

////////////////////////// import React from 'react';

export default function WaitlistManager({bookings}){

  const waiting=bookings.filter(b=>b.status==='waiting');

  return (<div><h3>Waitlist</h3><ul>{waiting.map(w=><li key={w._id}>{w.user.name} - party of
{w.partySize}</li>)}</ul></div>);
}

//////////////////////////

// client/src/components/SignUp.js

//////////////////////////

import React,{useState} from 'react';import axios from 'axios';

export default function SignUp(){const [form,setForm]=useState({});const
submit=()=>>axios.post('/api/auth/register',form).then(r=>alert(r.data.message));return(<div><input
placeholder="Name" onChange={e=>setForm({...form,name:e.target.value})}/><input
placeholder="Email" onChange={e=>setForm({...form,email:e.target.value})}/><input
type="password" placeholder="Password"
onChange={e=>setForm({...form,password:e.target.value})}/><button onClick={submit}>Sign
Up</button></div>);
}

//////////////////////////

// client/src/components/Login.js

//////////////////////////

import React,{useState} from 'react';import axios from 'axios';

export default function Login({onLogin}){const [creds,setCreds]=useState({});const
submit=()=>>axios.post('/api/auth/login',creds).then(r=>onLogin(r.data));return(<div><input
placeholder="Email" onChange={e=>setCreds({...creds,email:e.target.value})}/><input
type="password" placeholder="Password"
onChange={e=>setCreds({...creds,password:e.target.value})}/><button
onClick={submit}>Login</button></div>);
}

//////////////////////////

// client/src/components/Profile.js

```

```

//////////////////// import React
from 'react'; export default function
Profile({user}){
  return (<div><h2>Welcome, {user.name}</h2><p>Points: {user.points}</p></div>);
}

////////////////////
// client/src/components/OrderStatus.js
////////////////////
import React,{useEffect,useState} from 'react';import axios from 'axios';
export default function OrderStatus({orderId}){
  const [status,setStatus]=useState("");

  useEffect(()=>{axios.get(`/api/orders/${orderId}/status`).then(r=>setStatus(r.data.status));},[orderId]
);
  return (<div><h3>Order Status: {status}</h3></div>);
}

////////////////////
// client/src/components/NotificationList.js
////////////////////
import React from 'react';
export default function NotificationList({notifications}){
  return (<ul>{notifications.map(n=><li key={n.id}>{n.message}</li>)}</ul>);
}

////////////////////
// client/src/components/CheckoutPage.js
////////////////////
import React,{useState} from 'react';import { CardElement,useStripe,useElements } from
'@stripe/react-stripe-js';
export default function CheckoutPage({amount}){

```

```

const stripe=useStripe(),elements=useElements(),[msg,setMsg]=useState("");

const handle=async()=>{const {error,paymentIntent}=await
stripe.confirmCardPayment('CLIENT_SECRET',{payment_method:{card:elements.getElement(CardEle
ment)}});

  if(error)setMsg(error.message);else setMsg('Payment '+paymentIntent.status);
};

return (<div><CardElement/><button onClick={handle}>Pay
${amount}</button><p>{msg}</p></div>);
}

```

```

////////////////////

```

```

// client/src/components/Invoice.js

```

```

////////////////////

```

```

import React from 'react';import jsPDF from 'jspdf';

export default function Invoice({order}){

  const gen=()=>{const doc=new jsPDF();doc.text(`Invoice for Order
${order._id}`,10,10);order.items.forEach((i,idx)=>doc.text(`${i.quantity}x ${i.menuitem.name} -
${i.price}`,10,20+idx*10));doc.save(`invoice_${order._id}.pdf`);};

  return (<button onClick={gen}>Download Invoice</button>);
}

```

```

////////////////////

```

```

// client/src/components/AdminDashboard.js

```

```

////////////////////

```

```

import React,{useState,useEffect} from 'react';import axios from 'axios';

export default function AdminDashboard(){

  const [orders,setOrders]=useState([]),[menu,setMenu]=useState([]),[res,setRes]=useState([]);

  useEffect(()=>{axios.get('/api/orders').then(r=>setOrders(r.data));axios.get('/api/menu').then(r=>set
Menu(r.data));axios.get('/api/booking').then(r=>setRes(r.data));,[]);

  return (<div><h2>Admin Dashboard</h2><section><h3>Orders</h3>{orders.map(o=><div
key={o._id}><o._id> - {o.status}</div>)}</section><section><h3>Menu</h3>{menu.map(m=><div
key={m._id}><m.name></div>)}</section><section><h3>Reservations</h3>{res.map(r=><div
key={r._id}><r.user> - {r.status}</div>)}</section><section><h3>Support</h3><p>Email:
support@Elysian bites.com</p></section></div>);

```

```

}

////////////////////////////////////
// client/src/components/RatingInterface.js
////////////////////////////////////

import React,{useState} from 'react';import axios from 'axios';
export default function RatingInterface({userId}){

  const [rating,setRating]=useState(5),[comment,setComment]=useState("");
  const submit=()=>axios.post('/api/reviews',{userId,rating,comment});

  return (<div><h3>Rate Us</h3><select value={rating}
onChange={e=>setRating(e.target.value)}>[1,2,3,4,5].map(n=><option key={n}
value={n}>{n}</option></select><textarea value={comment}
onChange={e=>setComment(e.target.value)}></div><button onClick={submit}>Submit</button></div>);
}

////////////////////////////////////
// client/src/components/ReviewList.js
////////////////////////////////////

import React,{useEffect,useState} from 'react';import axios from 'axios';
export default function ReviewList(){const [reviews,setReviews]=useState([]);

  useEffect(()=>axios.get('/api/reviews').then(r=>setReviews(r.data)),[]);

  return (<ul>{reviews.map(r=><li key={r._id}>{r.user.name}: {r.rating}★ - {r.comment}</li>)}</ul>);
}

////////////////////////////////////
// client/src/components/PointsTracker.js
////////////////////////////////////

import React,{useEffect,useState} from 'react';import axios from 'axios';
export default function PointsTracker({userId}){

  const [points,setPoints]=useState(0);

  useEffect(()=>axios.get(`/api/loyalty/${userId}/points`).then(r=>setPoints(r.data.points)),[userId]);

  return (<div>Your Points: {points}</div>);
}

```

```

}

////////////////////////////////////
// client/src/components/CouponGenerator.js
////////////////////////////////////

import React,{useState} from 'react';import axios from 'axios';
export default function CouponGenerator({userId}){

  const [cost,setCost]=useState(0);

  const redeem=()=>axios.post('/api/loyalty/redeem',{userId,cost}).then(r=>alert(r.data.message));

  return (<div><h3>Redeem Points</h3><input type="number" value={cost}
onChange={e=>setCost(e.target.value)}/><button onClick={redeem}>Redeem</button></div>);
}

// Note: Additional styling, state management, routing, and context providers would tie these
components into App.js

////////////////////////////////////
// server/controllers/integratedController.js
////////////////////////////////////

const Reservation = require('../models/Reservation');
const Order = require('../models/Order');

exports.getUserDashboard = async (req, res) => {

  const userId = req.params.userId;

  const reservations = await Reservation.find({ user: userId, status: { $in: ['confirmed', 'waiting'] } });
  const orders = await Order.find({ user: userId });

  res.json({ reservations, orders });
};

////////////////////////////////////
// server/routes/integratedRoutes.js
////////////////////////////////////

```

```

const express = require('express');

const { getUserDashboard } = require('../controllers/integratedController');

const router = express.Router();

router.get('/user/:userId/dashboard', getUserDashboard);

module.exports = router;

////////////////////////////////////
// server/server.js (mount integrated routes)
////////////////////////////////////
// Add below existing mounts
app.use('/api/integrated', require('../routes/integratedRoutes'));

////////////////////////////////////
// client/src/components/IntegrationPage.js
////////////////////////////////////
import React, { useEffect, useState } from 'react';
import axios from 'axios';
import ReservationList from '../components/ReservationList';
import OrderList from '../components/OrderList';

export default function IntegrationPage({ userId }) {
  const [data, setData] = useState({ reservations: [], orders: [] });

  useEffect(() => {
    axios.get(`/api/integrated/user/${userId}/dashboard`).then(res => setData(res.data));
  }, [userId]);

  return (
    <div>

```



```
<h2>Your Reservations & Orders</h2>

<section>

  <h3>Reservations</h3>

  <ReservationList reservations={data.reservations} />

</section>

<section>

  <h3>Orders</h3>

  <OrderList orders={data.orders} />

</section>

</div>

);
}

// In client/src/App.js, add route:
// <Route path="/dashboard" render={() => <IntegrationPage userId={currentUser.id} />} />
```